



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

并行程序设计实验报告

GPU 编程实验

颜国庄

学号：2413140

年级：2024 级

专业：计算机科学与技术

指导教师：王刚

2026 年 6 月 19 日

目录

一、 问题分析	1
二、 并行算法设计	1
(一) 并行算法思路	1
(二) 整体数据结构与主流程	1
(三) cuBLAS 版本实现	2
(四) 手写 CUDA kernel 版本实现	4
(五) 正确性验证	6
三、 性能对比	7
四、 大模型使用情况	9
五、 代码链接	10
六、 HIP 课程学习	10

一、 问题分析

在原始 CPU 版本中, 虽然能够完成正确的上二对角化计算, 但在大规模矩阵下仍然会受到 CPU 核心数量、缓存容量和内存带宽的限制。GPU 拥有大量并行线程, 更适合处理矩阵子块中大量相似的元素更新任务。本次 GPU 编程实验的核心思路可以概括为在保持原有框架不变的前提下, 将 `to_bidiagonal()` 中最耗时的 GEMV / GER 部分搬到 GPU 上, 并比较 cuBLAS 和手写 kernel 的实现差异。

二、 并行算法设计

(一) 并行算法思路

每一轮左 Householder 变换时, 程序首先需要取得当前第 k 列从第 k 行开始的向量。由于矩阵 B 的最新数据已经保存在 GPU 端的 `dB` 中, 所以这里先从 GPU 中取出对应的列向量, 再拷回 CPU 端。随后 CPU 负责计算向量范数, 并按照原始 Householder 公式构造反射向量和系数 `beta`。这样处理没有改变原算法的数学逻辑, 也方便后续和 CPU 原始版本进行正确性对比。

当 Householder 向量构造完成并且有效时, 程序会根据编译时选择的版本, 分别调用 cuBLAS 实现或手写 kernel 实现来完成后续矩阵更新。左侧变换结束后, 再将第 k 列对角线以下的元素强制置零, 避免浮点误差导致本应为 0 的位置留下很小的残差。右 Householder 变换的整体流程与左侧类似, 只是处理对象变成第 k 行从第 $k+1$ 列开始的向量, 并在更新后清除超对角线右侧的元素。

这样设计保留了原始 Householder 上二对角化的整体流程, 只把其中计算量较大的矩阵运算交给 GPU 执行。优点是实现思路比较清楚, 和原 CPU 版本对应关系明显, 便于调试和验证; 同时, GEMV 和 GER 这类核心操作能够利用 GPU 的并行计算能力。缺点是当前实现还没有完全消除 CPU 和 GPU 之间的数据往返, 每一轮仍然需要传递 Householder 向量, 因此存在一定通信开销。后续如果继续优化, 可以考虑把向量构造过程也放到 GPU 上完成, 从而进一步减少每轮的数据传输。

(二) 整体数据结构与主流程

首先定义 CUDA/cuBLAS 错误检查宏。CUDA 接口统一通过 `CUDA_CHECK` 检查返回值; cuBLAS 版本额外定义了 `cublas_status_string()` 和 `CUBLAS_CHECK`, 用于在 cuBLAS 调用失败时输出更具体的错误位置和状态。这样比只看最终结果错误更容易定位问题。

```
1 #define CUDA_CHECK(call) do { ... } while (0)
2 #define CUBLAS_CHECK(call) do { ... } while (0)
```

性能统计由 `GpuProfile` 结构体保存：

```
1 struct GpuProfile {  
2     double h2d_init_ms = 0.0;  
3     double loop_total_ms = 0.0;  
4     double vec_d2h_ms = 0.0;  
5     double vec_h2d_ms = 0.0;  
6     double gpu_ops_ms = 0.0;  
7     double d2h_final_ms = 0.0;  
8 };
```

其中 `h2d_init_ms` 统计显存分配、输入矩阵上传、 U 和 V 初始化的时间；`loop_total_ms` 统计 Householder 外层循环总时间；`vec_d2h_ms` 统计每轮从 GPU 拷回当前列段或行段的时间；`vec_h2d_ms` 统计 Householder 向量上传时间；`gpu_ops_ms` 统计 cuBLAS 调用或手写 kernel 的 GPU 计算时间；`d2h_final_ms` 统计最终 B 、 U 、 V 拷回 CPU 的时间。

在正式计算前，主函数 `to_bidiagonal()` 会先进行数据和显存准备：

1. 先检查矩阵规模，若 $m \leq n$ ，直接抛出参数错误；
2. 在 CPU 端准备结果矩阵，以及存放奇异值和左右奇异向量的数组；
3. 将原矩阵从 `Matrix` 对象整理成连续的一维数组，方便后续拷贝到 GPU；
4. 在 GPU 端申请矩阵数据、左右变换矩阵、Householder 向量和临时工作区等空间；
5. 将整理后的矩阵数据从 CPU 拷贝到 GPU；
6. 初始化 GPU 端的左右变换矩阵，使其从单位矩阵开始累积变换；
7. 如果启用了 cuBLAS，则额外创建对应的计算句柄；
8. 进入 Householder 外层循环，逐步将原矩阵化为双对角形式；
9. 计算结束后，把 GPU 端的矩阵和变换结果拷回 CPU，并重新写回矩阵对象。

在资源释放方面，代码在可能提前退出的位置显式释放已申请的显存和 cuBLAS 句柄。这样即使 CUDA 或 cuBLAS 调用中途失败，也能尽量避免显存和句柄残留，方便后续调试和重复运行。

(三) cuBLAS 版本实现

cuBLAS 版本通过 `-DSVD_GPU_IMPL=1` 启用。相比纯 CUDA kernel 版本，这一版主要把 Householder 更新中的矩阵向量乘和秩一更新交给 cuBLAS 完成，CPU 端只负责准备向量、发起调用以及记录必要的时间开销。

`Matrix` 类采用行主序, 而 cuBLAS 默认按列主序解释矩阵, 所以我们将行主序的 $rows \times cols$ 子矩阵, 等价看成 cuBLAS 视角下的列主序 $cols \times rows$ 矩阵。也就是说, cuBLAS 实际看到的是原子矩阵的转置视图。后续根据这个视图选择 `CUBLAS_OP_N` 或 `CUBLAS_OP_T`, 并把 leading dimension 设为原矩阵的列数, 就可以避免整块转置带来的额外开销。

左 Householder 更新时, 子矩阵的实际维度为 $(m-k) \times (n-k)$, 代码中记为 `rows=m-k`、`cols=n-k`。数学上需要先计算

$$w = v^T B_{sub} = B_{sub}^T v.$$

由于 cuBLAS 看到的正好是 B_{sub}^T , 所以这里可以直接按非转置形式做矩阵向量乘:

```
1 CUBLAS_CHECK(cublasDgemv(handle, CUBLAS_OP_N,
2                           cols, rows,
3                           &one,
4                           dB + static_cast<size_t>(k) * n + k, n,
5                           d_v, 1,
6                           &zero,
7                           d_w, 1));
```

随后需要完成

$$B_{sub} = B_{sub} - \beta v w^T.$$

放到 cuBLAS 的转置视图下, 就变成

$$B_{sub}^T = B_{sub}^T - \beta w v^T.$$

因此这里使用秩一更新, 并把维度顺序写成 `cols, rows`:

```
1 CUBLAS_CHECK(cublasDger(handle,
2                           cols, rows,
3                           &minus_beta,
4                           d_w, 1,
5                           d_v, 1,
6                           dB + static_cast<size_t>(k) * n + k, n));
```

左侧变换还需要同步累积到 U 。这里的思路和更新 B 一样: 先得到当前列空间上的中间向量, 再用一次秩一更新把变换累积到 U 的对应子块中。实现时仍然按转置视图处理, 只是矩阵尺寸换成了 U 的子矩阵尺寸。

右 Householder 更新时, 子矩阵变为 $B[k:m, k+1:n]$, 维度为 `rows=m-k`、`cols=n-k-1`。这一步数学上需要计算

$$w = B_{sub} v.$$

但 cuBLAS 看到的是 B_{sub}^T , 所以要通过转置选项把结果转回原来的数学方向:

```

1 CUBLAS_CHECK(cublasDgemv(handle, CUBLAS_OP_T,
2                       cols, rows,
3                       &one,
4                       dB + static_cast<size_t>(k) * n + (k + 1), n,
5                       d_v, 1,
6                       &zzero,
7                       d_w, 1));

```

后续对 B 和 V 的更新也沿用同样的处理方式：先明确 cuBLAS 当前看到的是转置视图，再决定矩阵维度顺序和是否需要转置操作。这样可以在不改变原有 Matrix 存储格式的前提下，复用 cuBLAS 的 `gemv` 和 `ger` 完成主要计算。

在计时方面，代码用 CUDA event 包住 cuBLAS 计算段，并将这部分时间累计到 `gpu_ops_ms`。因此 profiling 结果里的 `gpu_ops` 不是整个 GPU 版本的运行时间，而是 cuBLAS 实际计算段的事件计时；CPU 准备、数据上传和其他控制逻辑并不包含在这个字段中。

(四) 手写 CUDA kernel 版本实现

手写 CUDA kernel 版本通过 `-DSVD_GPU_IMPL=2` 启用。和 cuBLAS 版本相比，这一版没有调用库函数，而是把 Householder 更新拆成两类基本操作：一次矩阵向量乘得到中间向量，再用一次秩一更新改写对应子矩阵。左、右两侧的更新流程相同，只是矩阵向量乘的方向不同。

Householder 向量会优先放到 constant memory 中。当向量长度不超过 4096 时，kernel 直接从 `__constant__` 数组读取；如果向量更长，则退回到 global memory 中读取。利用了短向量反复访问时的缓存优势，也避免了向量过长导致越界或额外拷贝失败。kernel 内部只通过一个很薄的读取接口区分这两种情况：

```

1 __device__ __forceinline__ double hv_load(const double *v, int idx, bool
   use_const) {
2     return use_const ? c_householder[idx] : v[idx];
3 }

```

左 Householder 更新 B 时，第一步是计算

$$w = v^T B_{sub}.$$

这里每个 block 负责一个输出列，block 内线程沿行方向分段累加。各线程先把自己的局部和写入 shared memory，再得到该列对应的最终结果：

```

1 int col = blockIdx.x;
2 for (int i = tid; i < rows; i += blockDim.x) {
3     sum += hv_load(v, i, use_const) * B[(k + i) * n + (k + col)];
4 }
5 sdata[tid] = sum;

```

```

6  __syncthreads();
7
8  for (int stride = blockDim.x / 2; stride > 0; stride >>= 1) {
9      if (tid < stride) sdata[tid] += sdata[tid + stride];
10     __syncthreads();
11 }
12
13 if (tid == 0) w[col] = sdata[0];

```

得到 w 后, 再按

$$B_{sub} = B_{sub} - \beta v w^T$$

逐元素更新矩阵。这个阶段没有规约, 每个线程只负责一个位置, 因此直接使用二维线程块映射到子矩阵的 `row, col`:

```

1  if (row < rows && col < cols) {
2      B[(k + row) * n + (k + col)] -= beta * hv_load(v, row, use_const) * w[col];
3  }

```

左侧正交矩阵 U 的累积方式与更新 B 类似。先用当前 Householder 向量计算每一行对应的中间结果, 再通过二维线程把秩一更新写回 U 的对应子块。这样 B 的变化和 U 的累积保持一致, 后面才能恢复原矩阵分解关系。

右 Householder 的思路相同, 只是中间向量的计算方向变成

$$w = B_{sub} v.$$

因此 block 不再按列分配, 而是每个 block 负责子矩阵的一行, 线程沿列方向累加点积。随后仍然用二维线程完成秩一更新。矩阵 V 的累积也沿用同样的两步: 先算中间向量, 再更新对应子块。

为了让归约 kernel 适配不同规模的子矩阵, 代码会根据当前向量长度选择 block 内线程数, 最多 256 个线程, 最少 32 个线程:

```

1  static int reduce_threads(int len) {
2      int t = 1;
3      while (t < len && t < 256) t <<= 1;
4      return t < 32 ? 32 : t;
5  }

```

矩阵逐元素更新部分则固定使用 16x16 的二维线程块。这样处理比较直接: GEMV 阶段需要 shared memory 和规约, GER 阶段每个元素互不依赖, 只需要把二维 grid 覆盖到目标子矩阵即可。

手写版本还在每轮结束后显式清理理论上应为零的位置, 包括第 k 列对角线以下元素, 以及第 k 行从 $k + 2$ 开始的右侧元素。启动这些清零 kernel 前会先判断待处理元素个数是

否大于 0，避免在某些旧工具链或特殊运行环境下启动 0 大小 grid。

这一版暂时没有把 GEMV 和 GER 融合到同一个 kernel 中。融合后虽然可能减少一次 kernel 启动和中间向量读写，但点积结果需要先在 block 内得到，再协调后续矩阵更新，代码复杂度会明显增加。本实验的重点是先得到一个结构清楚、便于和 cuBLAS 版本对照的手写 baseline，因此把融合优化留作后续工作。。

(五) 正确性验证

我将代码下载到了本地进行运行，其中 gkh.cpp 文件还原为最初始状态，即还没有修改为 simd 版本。

编译 cuBLAS 版本通过 `nvcc -O2 -std=c++17 -DNOMINMAX -DSVD_GPU_IMPL=1 -Xcompiler "/EHsc /utf-8" main.cpp bidiagonalization_gpu_full.cu gkh.cpp -lcublas -o main_cublas.exe`。

cuBLAS 版本：

```
||V^T V-I||_F      : 3.0553e-15
diagonal structure error : 0
descending order error  : 0
nonnegative diagonal   : yes
time bidiagonalization(ms): 6.7327
time gkh iteration(ms)  : 0.0122
结果: PASS

=== 随机 1000x1000 ===
[GPU bidiagonalization cuBLAS] m=1000, n=1000, H2D/init=2.3711 ms, loop_total=558.933 ms, vec_D2H=107.896 ms, vec_H2D=14.8559 ms, gpu_ops=345.309 ms, final_D2H=3.9928 ms
converged           : yes
||A-U*S*V^T||_F     : 2.06502e-10
relative recon error : 3.5726e-13
||U^T U-I||_F       : 2.34907e-13
||V^T V-I||_F       : 2.37338e-13
diagonal structure error : 0
descending order error  : 0
nonnegative diagonal   : yes
time bidiagonalization(ms): 575.217
time gkh iteration(ms)  : 5428.73
结果: PASS

=====
随机种子基值: 20260408
总上二对角化耗时(ms): 740.88
总GKH迭代耗时(ms): 5428.78
通过: 5 / 5
D:\hahah>
```

编译手写 kernel 版本通过 `nvcc -O2 -std=c++17 -DNOMINMAX -DSVD_GPU_IMPL=2 -Xcompiler "/EHsc /utf-8" main.cpp bidiagonalization_gpu_full.cu gkh.cpp -lcublas -o main_kernel.exe`

手写 kernel 版本：


```

||V^T V-I||_F      : 2.09185e-15
diagonal structure error : 0
descending order error  : 0
nonnegative diagonal   : yes
time bidiagonalization(ms): 5.1204
time gkh iteration(ms)  : 0.0138
结果: PASS

=== 随机 1000x1000 ===
[GPU bidiagonalization handwritten-kernel] m=1000, n=1000, H2D/init=2.1019 ms, loop_total=883.055 ms, vec_D2H=98.7018 ms
vec_H2D=51.2661 ms, gpu_ops=670.911 ms, final_D2H=4.0047 ms
converged           : yes
||A-U*S*V^T||_F    : 2.0657e-10
relative recon error : 3.57379e-13
||U^T U-I||_F      : 2.33562e-13
||V^T V-I||_F      : 2.32368e-13
diagonal structure error : 0
descending order error  : 0
nonnegative diagonal   : yes
time bidiagonalization(ms): 898.889
time gkh iteration(ms)  : 5641.74
结果: PASS

=====
随机种子基值: 20260408
总上二对角化耗时(ms): 1043.47
总GKH迭代耗时(ms): 5641.8
通过: 5 / 5
D:\hahah>

```

代码都通过了，正确性没有问题！

三、性能对比

我选择三个版本进行性能测试对比：原 CPU 版本、SVD_GPU_IMPL=1 的 cuBLAS 版本、SVD_GPU_IMPL=2 的手写 kernel 版本。其中原 cpu 版本中的 gkh.cpp 和 bidiagonalization.cpp 文件都还原成了最初始状态。测试全部在本地进行。表中时间为总上二对角化耗时。

编译 cpu 版本通过 `cl /O2 /EHsc /std:c++17 /utf-8 /DNOMINMAX main.cpp bidiagonalization.cpp gkh.cpp /Fe:main_cpu.exe`，随后在运行时指定种子值。

表 1: CPU、cuBLAS 与手写 kernel 性能对比

种子值	CPU 时间/ms	cuBLAS 时间/ms	手写 kernel 时间/ms
20260408	2464.58	678.61	1015.39
20260409	2580.91	672.71	1029.16
20260410	2476.71	682.53	1005.04
20260411	2405.47	642.66	994.13
20260412	2446.4	624.51	989.55

从平均效果看，CPU 版本平均耗时约为 2474.81 ms，cuBLAS 版本平均耗时约为 660.20 ms，手写 kernel 版本平均耗时约为 1006.65 ms。相比 CPU 原始版本，cuBLAS 版本和手写 kernel 版本都有了加速。

cuBLAS 版本明显快于手写 kernel 版本，说明对于 GEMV、GER 这类基础线性代数运算，cuBLAS 的底层优化更充分。手写 kernel 虽然利用了 GPU 并行能力，因此也明显快于

CPU 版本，但在访存优化、线程调度和归约效率方面仍然不如成熟的 cuBLAS 库。

本次实验中各版本的运行时间相比前几次实验整体减少较多。个人认为可能与本次测试改为本地运行有关：本地环境中任务调度和资源占用相对可控，不会像共享服务器那样受到其他用户任务、队列调度、CPU 负载波动和内存带宽竞争等因素影响，因此运行时间更加稳定。另一方面，本地测试使用的是独立显卡，GPU 版本能够直接利用本地 GPU 资源完成 GEMV/GER 运算，因此在上二对角化阶段表现出更明显的加速效果。因此后续性能比较方面主要基于同一台本地机器上的 CPU 原始版本、cuBLAS 版本和手写 kernel 版本进行，而不再直接使用服务器运行结果作为严格加速比依据。

Profiling 与通信开销分析

我们在标准错误流中输出 GPU 版本的 profiling 信息。cuBLAS 版本和手写 kernel 版本分别用不同前缀区分：

```
1 [GPU bidiagonalization cuBLAS]
2 [GPU bidiagonalization handwritten-kernel]
```

一次完整输出大致包含以下几类时间：

```
1 m=..., n=..., H2D/init=... ms, loop_total=... ms,
2 vec_D2H=... ms, vec_H2D=... ms, gpu_ops=... ms,
3 final_D2H=... ms
```

其中，H2D/init 记录初始显存申请、输入矩阵上传和单位矩阵初始化的开销；loop_total 记录整个 Householder 主循环时间；vec_D2H 和 vec_H2D 分别对应 Householder 向量构造过程中 CPU 与 GPU 之间的往返拷贝；gpu_ops 只统计 GEMV、GER 等核心 GPU 运算段；final_D2H 则是最后把 B 、 U 、 V 回传到 CPU 的时间。

表 2: Profiling 输出记录（单位：ms）

版本	H2D/init	loop_total	vec_D2H	vec_H2D	gpu_ops	final_D2H
cuBLAS	2.2224	462.807	95.3389	12.7066	316.757	4.0331
手写 kernel	1.9917	848.579	98.2128	43.5138	658.907	5.6786

从这组数据看，初始上传和最终回传都不是主要开销。两种实现的 H2D/init 和 final_D2H 也只有几毫秒，和主循环相比占比很小。真正拉开差距的是循环内部，尤其是 gpu_ops 这一项。

两种版本的 loop_total 和 gpu_ops 差距都稍大。也就是说，手写版本慢出来的主要部分并不是数据初始化或结果回传，而是 GEMV、GER 这些核心矩阵操作本身。这个结果也比较符合预期：上二对角化中大量操作都属于矩阵向量乘和秩一更新，cuBLAS 在归约、

访存、线程组织和底层算子调度上已经做了较多优化，手写 baseline 很难在第一版就达到同等效率。

另外，手写 kernel 的 `vec_H2D` 高于 cuBLAS 版本，说明 Householder 向量上传在手写版本中也带来了一定额外开销。不过从总耗时看，主要问题应该集中在 GPU 运算段。

这组 profiling 和前面的总耗时测试是吻合的：cuBLAS 版本整体更快，优势主要来自循环中的核心计算；手写 kernel 版本虽然保留了清晰的并行结构，也相比 CPU 版本有加速，但 GEMV 归约和 GER 更新还比较基础。后续优化可以优先考虑两点：一是减少 Householder 向量在 CPU 和 GPU 之间的往返传输，二是尝试把 GEMV 和 GER 融合，减少中间向量读写和 kernel 启动次数。

四、大模型使用情况

本次实验过程中使用了 AI 工具进行辅助，主要用于以下几个方面：

首先，使用 AI 辅助对部分文字表述进行了润色，使问题分析、算法流程、性能分析等内容更加连贯，减少口语化或重复表述。对于 cuBLAS 版本和手写 CUDA kernel 版本的实现说明，AI 主要用于帮助整理叙述顺序，使报告能够更清楚地说明行主序矩阵与 cuBLAS 列主序视角之间的关系，以及 GEMV、GER 在 Householder 更新中的作用。

在代码说明部分，AI 辅助将部分实现细节转化为更适合实验报告的描述，例如显存申请与释放、Householder 向量上传等。对相关内容均结合实际代码逻辑进行整理。

另外在性能分析方面，使用 AI 辅助对实验结果进行了文字归纳，包括比较 CPU 版本、cuBLAS 版本和手写 kernel 版本的运行时间差异，解释主要耗时集中在 Householder 主循环和 GPU 核心计算段的原因，并对后续可能的优化方向进行了表述整理。

实验中的算法选择、代码实现、编译运行、正确性验证、性能测试数据记录以及最终结论判断均基于本次实验过程完成。AI 工具主要承担文字润色、结构整理和表达优化的辅助作用。

Q：如何解释 cuBLAS 默认列主序与本工程 `Matrix` 类行主序之间的关系使其在报告上更简洁？

A：可以将行主序的 $rows \times cols$ 子矩阵理解为 cuBLAS 视角下的列主序 $cols \times rows$ 转置视图。这样不需要显式转置整块矩阵，只要根据实际数学操作选择 `CUBLAS_OP_N` 或 `CUBLAS_OP_T`，并正确设置 leading dimension，就可以复用 cuBLAS 的 GEMV 和 GER 操作。

Q：根据 profiling 数据如何说明 cuBLAS 版本比手写 kernel 版本更快？

A：从 profiling 数据看，两种实现的初始数据上传和最终结果回传耗时都较小，主要差距集中在 `loop_total` 和 `gpu_ops`。这说明性能差异主要来自 Householder 主循环中的

GEMV 和 GER 计算。cuBLAS 在访存、归约和线程调度方面优化更成熟，因此核心 GPU 运算段耗时低于手写 kernel 版本。

五、 代码链接

GitHub 链接：<https://github.com/Yanguozhuang/parallel>

六、 HIP 课程学习

lab1

lab1 主要学习 GPU 编程的基础模型，包括 host 与 device 的区别、kernel 的启动方式，以及 thread、block、grid 的层次结构。通过简单的一维并行任务，理解如何用 blockIdx.x、threadIdx.x 和 blockDim.x 计算全局线程编号，并掌握把数据拷贝到 GPU、执行 kernel、再拷回结果的基本流程。

下面是代码填空的图片和最终生成的两张图：

Exercise 1: GPU Specifications

Based on the output above, record your GPU's specifications:

Property	Your GPU Value	Significance
Device Name	__ AMD Radeon 890M Graphics __	GPU model identifier
Compute Units	8	Number of parallel execution units
Wavefront Size	32	Threads per warp (32 on RDNA , 64 on CDNA)
Max Threads per Block	1024	Upper limit for blockDim
Max Shared Memory per Block	64KB	LDS available per block

Question: How many total threads can your GPU execute simultaneously if all CUs are fully utilized?

Your calculation: **16384threads**__

Kernel Structure

```
__global__ void vector_add(const float* A, const float* B, float* C, int N) {  
    // Step 1: Calculate global thread index  
    int idx = _____blockIdx.x * blockDim.x + threadIdx.x_____; // Fill in  
  
    // Step 2: Boundary check (important when N is not divisible by block size)  
    if (____idx < N____) { // Fill in the condition  
        // Step 3: Perform computation  
        C[idx] = _____A[idx] + B[idx]_____; // Fill in  
    }  
}
```

六、HIP 课程学习

Exercise 3: Launch Configuration Calculation

Given `N = 1000` elements, complete the table:

Block Size	Grid Size Formula	Grid Size	Total Threads	Idle Threads
64	<code>ceil(1000/64)</code>	16	1024	24
128	<code>ceil(1000/128)</code>	8	1024	24
256	<code>ceil(1000/256)</code>	4	1024	24
512	<code>ceil(1000/512)</code>	2	1024	24

Question: Which block size gives the best thread utilization? Why might this not always be the best choice?

Your answer: 四种 block size 的线程利用率相同, 都是 $1000 / 1024 = 97.66\%$ 。但这不一定代表性能最好, 因为实际性能还取决于 wavefront 对齐、occupancy、内存带宽和资源使用情况。

Exercise 4: Analyze Your Results ¶

Based on the multi-trial results above:

1. Which block size achieved the **lowest mean execution time**? 256
2. Which block size achieved the **highest memory bandwidth**? 256
3. Are the differences between block sizes **statistically significant** (look at error bars)? 不太显著, 因为各个 block size 的误差棒有较多重叠, 平均时间差距也很小, __
4. Why does Vector Add performance depend more on bandwidth than on block size?

Your answer: 向量加法的性能更多取决于内存带宽, 而不是 block size, 因为每个元素只进行一次浮点加法, 但需要从全局内存读取 A 和 B 两个值, 并把结果写回 C, 也就是两次读和一次写, 内存访问开销远大于计算开销。因此这个 kernel 是 memory-bandwidth-bound, 只要 block size 已经和 wavefront 对齐, 并且 occupancy 足够, 继续改变 block size 对性能影响就比较小。

Exercise 5: Wavefront Analysis

Question 1: A kernel is launched with block size 100. How many wavefronts does each block require? How many lanes are wasted?

- Wavefronts per block: 4 (hint: $\lceil 100/32 \rceil$)
- Wasted lanes: 28 (hint: wavefronts $\times$ 32 - 100)

Question 2: If the same kernel were compiled with `-mwavefrontsize64`, how would the answers change?

- Wavefronts per block (wave64): 2 (hint: $\lceil 100/64 \rceil$)
- Wasted lanes (wave64): 28__

Question 3: What happens when threads within a wavefront take different branches (if-else)?

Your answer: 当同一个 wavefront 内的线程进入不同的 if-else 分支时, 会发生分支发散。GPU 会把不同分支路串行执行, 并用掩码控制哪些线程参与执行, 因此部分 lane 会空闲, 导致执行效率下降。

Exercise 6: Occupancy Calculation

For **Radeon 8060S (RDNA 3.5)**: 20 CUs, Warp Size = 32, Max Blocks per CU = 64, Max Threads per Block = 1024

To calculate occupancy, you need to know the max wavefronts per CU (check `rocminfo` output).

Block Size	Wavefronts/Block	Blocks that fit	Active Wavefronts	Notes
32	1	limited by max blocks (64)	64	Full occupancy
64	2	32	64	Full occupancy
128	4	16	64	Full occupancy
256	8	8	64	Full occupancy
512	16	4	64	Full occupancy
1024	32	2	64	Full occupancy

Exercise 7: Analyze Sub-Wavefront Results

Based on the multi-trial results above:

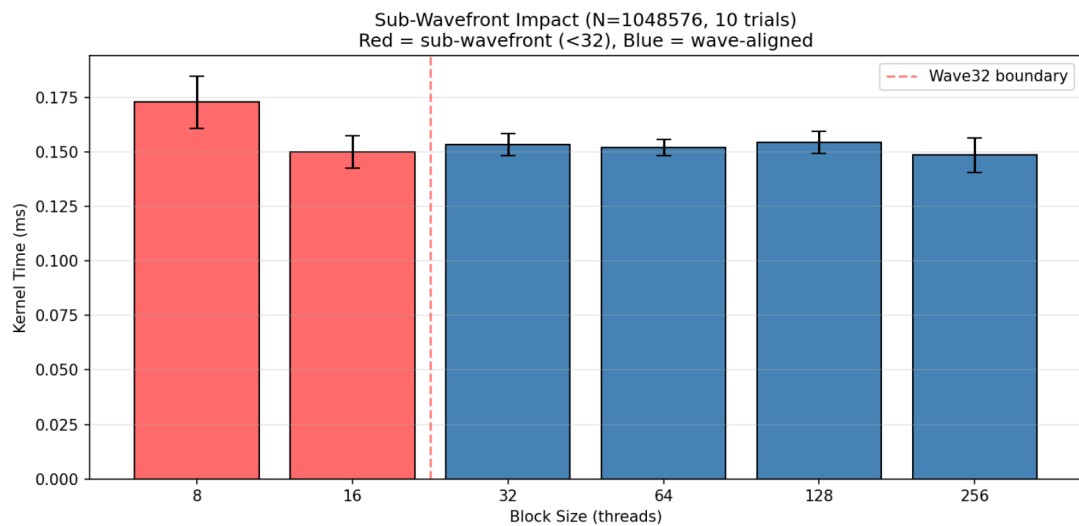
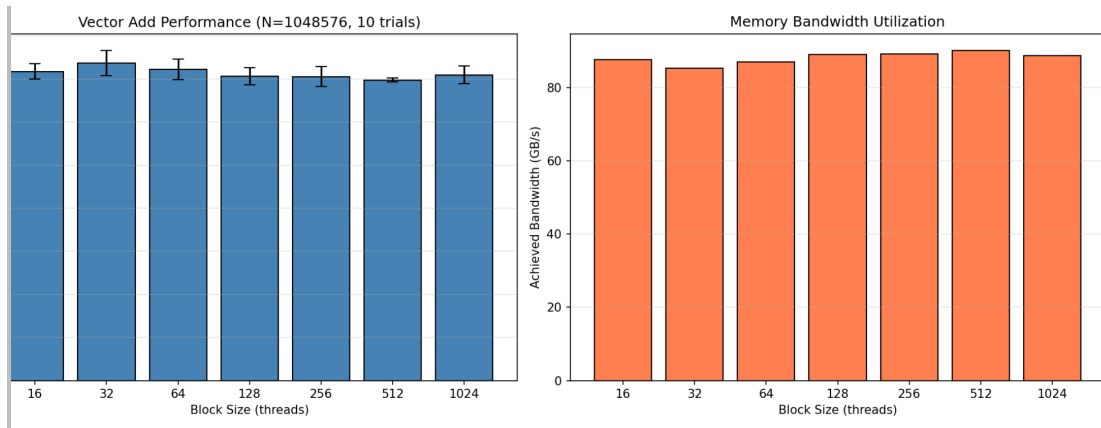
- Mean time for block size 8 (sub-wave): 0.1649 ms
- Mean time for block size 16 (sub-wave): 0.1518 ms
- Mean time for block size 32 (wave-aligned): 0.1435 ms
- Mean time for block size 256 (large): 0.1453 ms

Question 1: Is there a noticeable performance penalty for sub-wavefront block sizes (8, 16) compared to wave-aligned sizes (32, 64, 128)?

Your answer: 是的, block size 为 8 和 16 时属于 sub-wavefront, 平均执行时间分别为 0.1649 ms 和 0.1518 ms, 高于 wave-aligned 的 32 和 64, 因此存在一定性能损失, 尤其是 block size 为 8 时更明显, 因为每个 wavefront 中有较多 lane 被浪费。__

Question 2: On this RDNA 3.5 GPU, the wavefront size is 32. If you were on an MI100 (wavefront = 64), which block sizes would become sub-wavefront?

Your answer: 如果在 MI100 上, wavefront size = 64, 那么所有小于 64 的 block size 都是 sub-wavefront, 因此在本实验测试的 block sizes 中, 8、16 和 32 会变成 sub-wavefront,



lab2

Lab2 学习了矩阵转置问题，重点是二维线程索引和二维数据在线性内存中的存储方式。通过 naive transpose 理解 row-major 布局下读取通常是连续的，但写入转置矩阵时会产生 strided access，导致 memory coalescing 变差。进一步学习使用 shared memory tiling，把数据先按 tile 读入 shared memory，再转置写回，从而改善访存效率。

Exercise 1: 2D Indexing

Given `blockDim = (32, 32)`, calculate the global indices:

blockIdx	threadIdx	Global (idx, idy)
(0, 0)	(5, 10)	(5, 10)
(1, 0)	(0, 0)	(<u>32</u> , 0)
(2, 3)	(15, 20)	(79, 116)

Question: For a matrix with `rows=100`, `cols=200`, how many blocks are needed?

Your calculation: **28**

Exercise 2: Understand the Index Transformation

For a 4×6 matrix (rows=4, cols=6), fill in the table:

Thread (idx, idy)	Input Index	Output Index
(0, 0)	$0*6+0 = 0$	$0*4+0 = 0$
(1, 0)	$1*6+0 = 6$	$1*4+0 = 4$
(0, 1)	$0*6+1 = 1$	$0*4+1 = 1$
(3, 2)	$3*6+2 = 20$	$3*4+2 = 14$

Question: Why is the write index formula `idx * rows + idy` instead of `idx * cols + idy`?

Your answer: 因为输出矩阵是转置后的矩阵，尺寸是 `cols × rows`。在线性 row-major 存储中，`output[idx][idy]` 的行号是 `idx`，列号是 `idy`，而输出矩阵每一行长度是 `rows`，所以线性下标应为 `idx * rows + idy`，而不是 `idx * cols + idy`。

Exercise 3: Performance Analysis

Record execution times:

Test	Dimensions	Elements	Time
1	16×16	256	0.0077ms
2	128×32	4,096	0.0107ms
3	1×1024	1,024	0.0085ms
4	1001×2001	2,003,001	0.2384ms

Question: The large test has ~500x more elements than test 2. Is the time 500x longer? Why or why not?

Your answer: 不是。Test 4 的元素数量约为 Test 2 的 489 倍，但运行时间只是 $0.2384 / 0.0107 \approx 22.3$ 倍，并不是 500 倍左右。原因是小规模测试主要受 kernel launch overhead 影响，GPU 并没有被充分利用；而大规模测试中并行度更高，能更充分利用 GPU 的计算资源和内存带宽。不过 naive transpose 的写入是 strided write，属于非合并访问，因此大测试的性能仍然受到内存访问模式限制。

Exercise 4: Coalescing Analysis

For a 1024×1024 matrix with 32×32 thread blocks (one wave32 = one row of the block):

Question 1: When reading `input[idy * cols + idx]`, are consecutive threads (in `threadIdx.x`) reading consecutive addresses?

Your answer: 是的。对于 `input[idy * cols + idx]`，同一个 wavefront 中连续的 `threadIdx.x` 会让 `idx` 连续增加，因此访问的是连续的内存地址，所以读取是合并访问 coalesced access。

Question 2: When writing `output[idx * rows + idy]`, what is the stride between addresses written by consecutive threads?

Your answer: 连续线程写入 `output[idx * rows + idy]` 时，`idx` 每增加 1，写入地址就增加 `rows` 个 float。对于 1024×1024 矩阵，stride 是 1024 个 float，也就是 $1024 \times 4 = 4096$ bytes。

Question 3: If each memory transaction fetches 128 bytes (32 floats), how many transactions are needed for 32 threads (one wavefront) to write non-coalesced?

Your answer: 需要 32 次 memory transactions。因为 32 个线程写入的地址彼此相隔 4096 bytes，不能合并成一次连续的 128-byte transaction，所以基本上每个线程 都需要一次独立的内存事务。

Exercise 5: Tiled Transpose Analysis ¶

Question 1: Why do we add `+1` to the shared memory declaration (`tile[BLOCK_SIZE][BLOCK_SIZE + 1]`)?

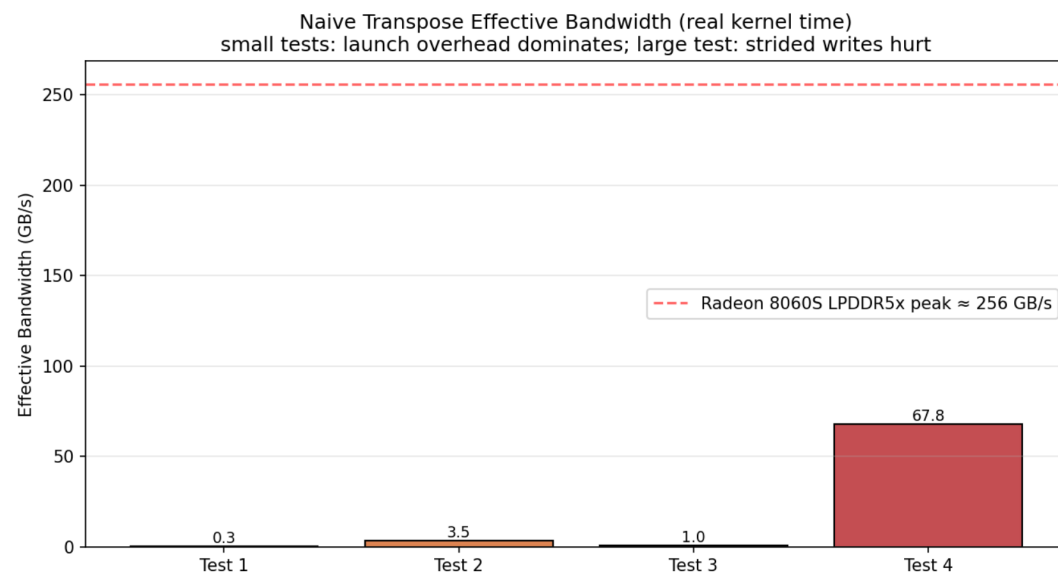
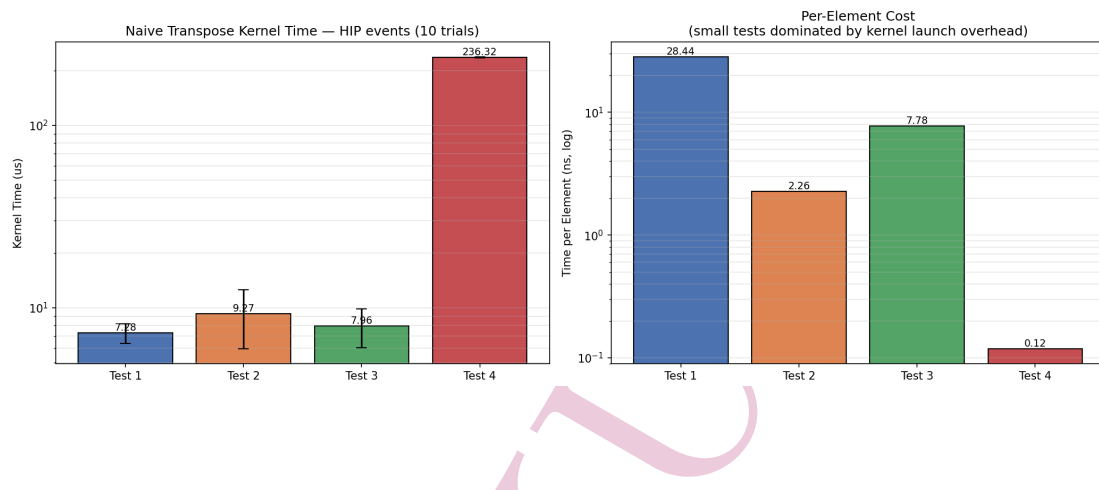
Your answer: 加 `+1` 是为了避免 `shared memory bank conflicts`。转置时线程会按列访问 `shared memory`。如果 `tile` 是 `32x32`，多个线程可能访问到同一个 `bank`，导致 `bank conflict`。把第二维改成 `BLOCK_SIZE + 1` 后，每一行的步长发生变化，可以让访问分散到不同 `bank` 中，从而减少冲突。

Question 2: In the write step, why do we swap `blockIdx.x` and `blockIdx.y`?

Your answer: 因为转置后，原来位于输入矩阵中 `blockIdx.x`、`blockIdx.y` 位置的 `tile`，应该写到输出矩阵中 `blockIdx.y`、`blockIdx.x` 的位置。交换 `blockIdx.x` 和 `blockIdx.y` 可以把每个 `tile` 写到转置后的正确位置。_____

Question 3: How much shared memory does each block use for `BLOCK_SIZE=32`?

Your calculation 4224: _____ bytes



lab3

Lab3 学习了直方图 histogram 的并行实现。核心问题是多个线程可能同时更新同一个 bin，产生 race condition，因此需要 atomic 操作。通过比较 naive GPU 方法和优化方法，理解为什么重复扫描输入数组会造成大量全局内存访问，以及如何用 shared memory 先在 block 内局部统计，再合并到 global histogram，从而减少 global atomic 的开销。

Exercise 1: Analyze the Problem

Question 1: How many times is each input element read from global memory?

Your answer: 每个 input 元素会被读取 num_bins 次。因为 naive 方法中每个 block 负责一个 bin，而每个 block 都会扫描完整的输入数组。__

Question 2: If num_bins = 256 and N = 10M, what is the total global memory traffic?

Your calculation: 10.24GB_24 GB__

Question 3: Why is this approach inefficient?

Your an这种方法效率很低，因为每个 bin 都要重新扫描一遍完整输入数组，导致同一个 input 元素被重复读取 num_bins 次。全局内存访问量从 O(N) 变成 O(N × num_bins)，内存带宽浪费严重，所以当 N 和 num_bins 较大时会非常慢。swer: ____

Exercise 2: Understand the Optimization

Question 1: Why is atomicAdd on shared memory faster than on global memory?

Your answer: shared memory 位于 GPU 片上，访问延迟低、带宽高；global memory 位于片外，访问延迟更高。先在 shared memory 中执行 atomicAdd，可以减少慢速 global atomicAdd 的次数，因此更快。

Question 2: In the merge step, how many global atomicAdd operations occur per bin?

Your answer: merge step 中，每个 block 会对每个 bin 执行一次 global atomicAdd，所以每个 bin 的 global atomicAdd 次数等于 block 的数量，也就是 gridDim.x 次。__

Question 3: What is the purpose of the grid-stride loop pattern `for (int i = idx; i < N; i += totalthread)`?

Your answer: grid-stride loop 的作用是让所有线程共同遍历完整输入数组。即使 N 很大、线程总数有限，每个线程也可以通过 i += totalthread 继续处理后面的元素，从而覆盖完整输入，同时保持较好的并行性。____

Exercise 3: Record Results

Fill in the execution times:

Implementation	Time (testcase 2)	Time (testcase 4)
Serial (CPU)	2.4992 ms	21.9634 ms__
Naive GPU	21.5106 ms	2572.8780 ms
Optimized GPU	0.4575 ms	4.4492 ms

Calculate speedups:

- Optimized vs Serial: __4.94__ x
- Optimized vs Naive: __578.28__ x

Exercise 4: Memory Efficiency

Question: Calculate the memory traffic reduction ratio:

$$\text{Reduction} = \frac{\text{Naive reads}}{\text{Optimized reads}} = \frac{N \times \text{num_bins}}{N} = ?$$

Your answer: **num_bins**

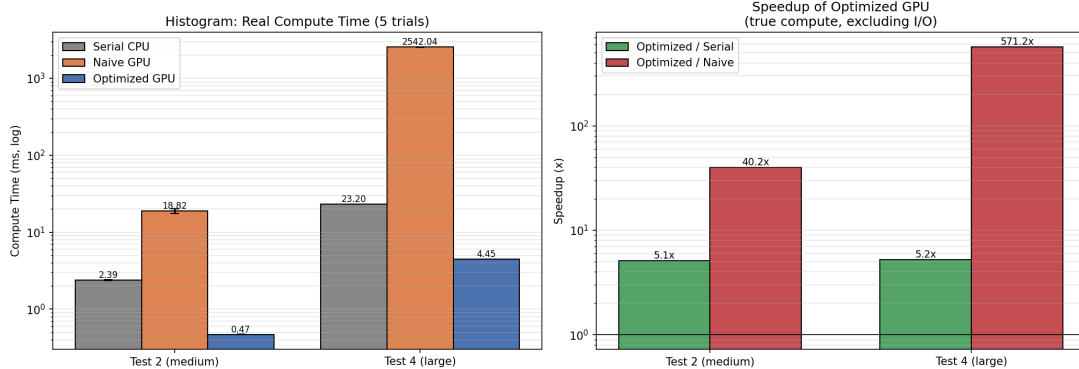
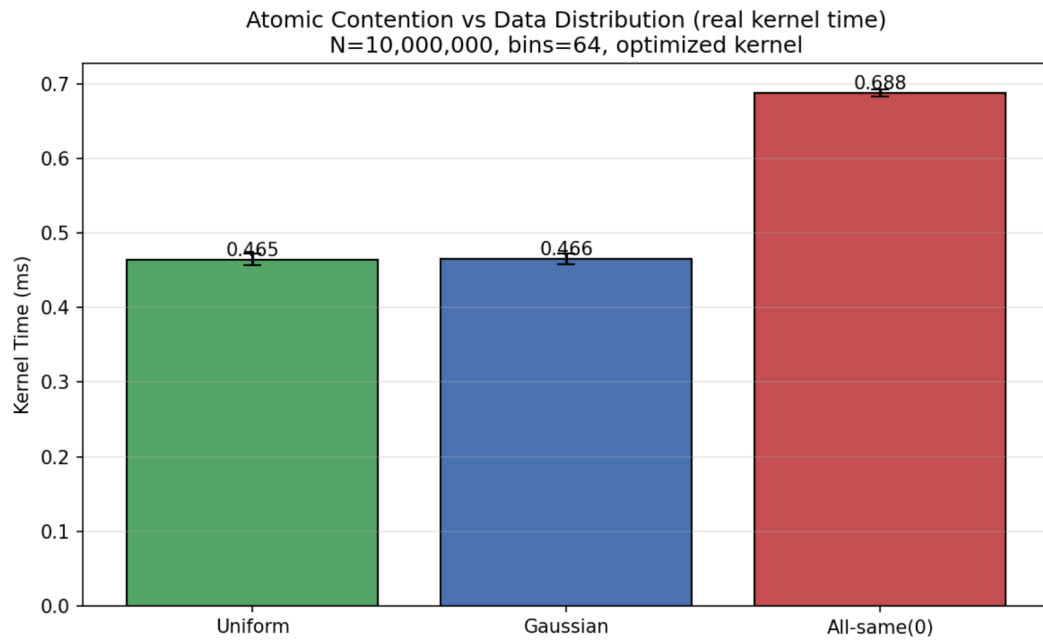
Exercise 5: Contention Scenarios

Scenario 1: All input values are the same (e.g., all zeros)

- What happens to performance? 性能会变差
- Why? 因为所有输入值都相同，很多线程会同时对同一个 histogram bin 执行 atomicAdd，产生严重 atomic contention，即使 atomic 操作发生在 shared memory 中，多个线程竞争同一个地址也会导致操作被串行化，从而降低性能。

Scenario 2: Input values are uniformly distributed across all bins

- Expected contention level: 较低。
- Why? 因为输入值均匀分布在所有 bins 上，线程的 atomicAdd 会分散到不同的 histogram bin 中，不会集中竞争同一个位置，所以 contention 较低，性能通常更好。



lab4

Lab4 学习了 parallel reduction, 也就是把大量数据规约成一个结果, 例如求和、最大值、最小值等。先分析 naive atomic reduction 的性能瓶颈, 再学习 tree-based reduction, 用 shared memory 在 block 内逐步合并部分结果。还了解 warp shuffle 优化, 它可以在一个 warp 内直接交换寄存器数据, 减少 syncthreads() 开销。同时分析 occupancy、block size、atomic 次数和实际性能之间的关系。

六、HIP 课程学习

Exercise 1: Understand Tree Reduction

Question 1: For an array of 1024 elements with block size 256, how many reduction steps occur within each block?

Your answer: **8**

Question 2: After block-level reduction, how many partial sums remain?

Your answer: **4**

Question 3: Why do we use `__syncthreads()` after each reduction step?

Your answer: 因为每一步 reduction 都依赖上一步的结果。__syncthreads() 可以保证同一个 block 中的所有线程都完成当前 stride 的加法之后，再进入下一轮 reduction。否则有些线程 可能会读取到还没有更新完成的数据，导致结果错误。__

Exercise 2: Analyze Occupancy Results

From the test output, fill in the table:

Block Size	Warps/Block	Occupancy (%)	Execution Time (ms)
64	2	100%	_____
128	4	100%	_____
256	8	100%	105 _____
512	16	100%	_____
1024	32	100%	_____

Question: Does higher occupancy always mean better performance?

Your answer: **No. Higher occupancy does not always mean better performance, because synchronization, memory access pattern, warp efficiency, shared memory usage, and atomic contention also affect reduction performance.**__

Exercise 3: Think About Trade-offs ¶

Question 1: With block size 64, how many final atomic operations occur for N = 1M elements?

Calculation: $\text{grid_size} = \text{ceil}(1,000,000 / 64) = \mathbf{15625}$

Question 2: With block size 1024, how many final atomic operations occur?

Calculation: $\text{grid_size} = \text{ceil}(1,000,000 / 1024) = \mathbf{977}$

Question 3: Why might fewer atomic operations improve performance?

Your answer: **更少的 atomic operations 可以减少多个 block 同时更新同一个 global output 时的竞争。global atomicAdd 会产生较高开销和串行化，atomic 次数越少，contention 越低，因此性能可能更好。**

Exercise 4: Algorithm Analysis

Question 1: In tree reduction, what fraction of threads are idle after the first step?

Your answer: **50%**

Question 2: Why does warp shuffle not require `__syncthreads()` ?

Your answer: __warp shuffle 是同一个 warp / wavefront 内线程之间的寄存器级数据交换。warp 内线程 本来就是同步执行的，因此不需要像 shared memory 跨整个 block 那样使用 **syncthreads()**。

Question 3: How does multi-element per thread improve memory bandwidth utilization?

Your answer: multi-element per thread 让每个线程先从 global memory 中读取并累加多个元素，再参与 block 内 reduction。这样可以减少 block 数量和最终 global atomicAdd 次数，同时让内存读取更连续、更充分利用带宽，因此能提高 memory bandwidth utilization。____

```
%%bash
echo "=== Benchmark with N=1M (testcases/2.in) ==="
./exe_benchmark testcases/2.in
```

```
=== Benchmark with N=1M (testcases/2.in) ===
=== Reduction Benchmark ===
Device: AMD Radeon 890M Graphics
N = 1000000, Block size = 256
```

Strategy	Time (ms)	Result
0. Naive Atomic	0.0847	-74993790
1. Tree (Shared Memory)	0.0640	-74993790
2. Tree + Warp Shuffle	0.0631	-74993790
3. Multi-Element + Tree + Shuffle	0.0479	-74993790

```
=== Speedup vs Naive ===
Tree:          1.32x
Warp Shuffle:  1.34x
Multi-Element: 1.77x
```

```
Verification: ALL PASSED (all strategies produce same result)
```

lab5

Lab5: 学习了 Monte Carlo 数值积分，把大量采样点的函数值并行累加来估计积分结果。实验中先使用每个线程计算一个样本贡献并通过 atomicAdd 累加到全局结果的方法，分析其 atomic contention 瓶颈。随后学习用 shared memory reduction 或两阶段归约来减少 global atomic 次数，并讨论 double 与 float 在大量样本累加时的精度差异，以及样本数增加时性能和误差的变化。

Exercise 1: Understand the Algorithm

Question 1: Why is the division by `n_samples` done in each thread instead of once at the end?

Your answer: 因为每个线程先计算自己对样本对最终积分的贡献: $tmp = (b - a) * y_samples[id] / n_samples$ 。

这样每个线程 atomicAdd 的就是已经归一化后的积分贡献值，最后 result 中直接就是积分结果。当然，也可以先累加所有 y 值，最后再统一乘以 $(b - a) / n_samples$ ；这样通常更适合优化版本。___

Question 2: What is the time complexity of the atomic reduction approach?

Your answer: 时间复杂度是 $O(n_samples)$ ，因为每个样本被一个线程处理一次。

但是由于所有线程都对同一个 result 执行 atomicAdd，会产生严重 atomic contention，所以实际性能会受到原子操作串行化的影响。___

Question 3: How could you modify this kernel to use shared memory reduction instead of atomics?

Your answer: 可以让每个 block 先在 shared memory 中做局部 reduction，每个线程把自己的 y 值放入 shared memory，然后通过 tree reduction 得到该 block 的局部和。最后只让每个 block 的 thread 0 对全局 result 执行一次 atomicAdd。这样可以把 global atomicAdd 次数从 $n_samples$ 次减少到 block 数量次。er: ___

Exercise 2: Manual Calculation

For Test 1, verify the result manually:

Given: $a = 0, b = 2, n = 8$ samples

$$\text{Formula: result} = \frac{(b-a)}{n} \sum_{i=1}^n y_i = \frac{2}{8} \times \sum y_i$$

Your calculation:

- Sum of y values: **23.7507**
- Expected result: **5.937675**
- Does it match the program output? **Yes**



Exercise 3: Scalability Analysis

Record the execution times:

Test	Sample Count	Execution Time
4	100,000	0.099s
7	10,000,000	0.958s
8	100,000,000	8.438s

Question 1: When sample count increases 100x (from test 4 to 7), how much does execution time increase?

Your answer: 9.68倍

Question 2: Is the scaling linear? Why or why not?

Your answer: 不是严格线性。样本数从 100,000 增加到 10,000,000, 增加了 100 倍, 但运行时间只从 0.099 s 增加到 0.958 s, 大约增加 9.6 倍。原因是小规模测试中固定开销占比较大, 例如程序启动、文件读取、GPU 初始化和数据拷贝; 随着样本数变大, GPU 并行度更高, 计算资源利用更充分, 所以时间不会严格按样本数线性增长。同时 naive 版本中 atomicAdd 竞争也会影响大规模情况下的性能。

Exercise 4: Bottleneck Analysis

Question 1: If the computation takes 1 cycle and atomic takes 100 cycles, what is the theoretical efficiency for $N=1M$ threads?

Hint: Total work = $N \times (\text{compute} + \text{atomic})$

Your calculation: **0.99%**

Question 2: How would shared memory reduction improve this?

Your answer: shared memory reduction 可以让每个 block 先在 shared memory 中把多个线程的结果合并, 最后每个 block 只执行一次 global atomicAdd, 这样可以大幅减少 global atomic 的次数, 降低对同一个 result 的竞争, 从而提高性能。

Question 3: With 256 threads per block and $N=1M$, how many global atomics would shared memory reduction require?

Your calculation: 3907

六、HIP 课程学习

Exercise 5: Precision Considerations

Question 1: The kernel uses `double` (64-bit). How many significant digits does double precision provide?

Your answer: *double precision 通常提供大约 15 到 16 位十进制有效数字。*

Question 2: If we switch to `float` (32-bit), what problems might occur with $N=100M$ samples?

Your answer: *如果改成 float, 只有大约 6 到 7 位十进制有效数字。对于 $N=100M$ 这种大量样本累加, 会产生明显的舍入误差, 小数贡献可能在大和中丢失, 最终积分结果精度下降。atomicAdd 的累加顺序不固定, 也会让浮点误差具有不确定性。*

Exercise 6: Optimization Analysis

Question 1: In the optimized version, how many global atomics occur for $N=1M$ with 256 blocks?

Your answer: *优化版本中每个 block 只执行一次 global atomicAdd。*

如果有 256 个 blocks, 则 global atomics = 256 次。 ____

Question 2: What is the purpose of the grid-stride loop `for (int i = idx; i < n_samples; i += blockDim.x * gridDim.x)?`

Your answer: *grid-stride loop 的作用是让有限数量的线程覆盖任意大的 $n_samples$ 。每个线程先处理 idx 位置的元素, 然后每次增加 $blockDim.x * gridDim.x$, 继续处理后面的元素。这样可以提高线程复用率, 并适合处理非常大的输入。 ____*

Question 3: Why is `(b - a) / n_samples` multiplication done only by thread 0?

Your answer: *因为 shared memory reduction 后, thread 0 已经拿到了该 block 的局部 sum。只让 thread 0 乘以 $(b - a) / n_samples$ 并执行 atomicAdd, 可以避免每个线程重复做 缩放计算, 也减少 global atomic 操作数量。 ____*

lab6

lab6 学习 K-Means 聚类算法在 GPU 上的并行化。每轮迭代包括两个阶段: assignment phase 中每个线程负责一个点, 计算它到所有 centroid 的距离并选择最近簇; update phase 中根据 labels 重新计算每个 centroid 的均值。实验重点包括距离计算的 $O(Nk)$ 开销、shared memory atomic 对 centroid 累加的优化、empty cluster 的处理、收敛检测, 以及使用 prev/new centroid 双缓冲避免读写冲突。

Exercise 1: Algorithm Analysis

Question 1: In the assignment kernel, what is the time complexity per thread if $k = 100$?

Your answer: *$O(100)$*

Question 2: Why does the recalculation kernel use shared memory atomics before global atomics?

Your answer: *recalculation kernel 先使用 shared memory atomics, 是为了在每个 block 内先局部累加 centroid 的 x 坐标和 y 坐标和以及点数量。shared memory 位于片上, 访问更快, 竞争开销比 global memory 小。最后每个 block 再把局部结果用 global atomics 合并到全局数组中, 这样可以减少慢速 global atomic 的次数, 提高性能。*

Question 3: What happens if a centroid has no points assigned to it (empty cluster)?

Your answer: *如果某个 centroid 没有任何点分配给它, 也就是 empty cluster, 那么 count = 0。代码中会保留它原来的位置:*

`new_centroids_x[idx] = prev_centroids_x[idx] new_centroids_y[idx] = prev_centroids_y[idx]`

这样可以避免除以 0, 也避免 centroid 变成无效值。 ____

Exercise 2: Performance Analysis

Record the execution time for test 4:

- Sample size: 50,000
- Clusters (k): 50
- Max iterations: 100
- Execution time: **0.127s** ____

Question: What is the dominant computation in each iteration?

Your answer: *每一轮迭代中最主要的计算是 assignment phase, 也就是每个点计算到所有 k 个 centroid 的距离, 并选择最近的 centroid。对于 N 个点和 k 个 clusters, 这一步需要 $N \times k$ 次距离计算, 时间复杂度是 $O(Nk)$ 。在 Test 4 中, 需要大约 $50,000 \times 50 = 2,500,000$ 次距离计算, 因此它通常是每轮迭代的主要开销。*

Exercise 3: Compute Intensity

For $N = 50,000$ points and $k = 50$ clusters:

Assignment kernel:

- Distance calculations per point: **50**
- Total distance calculations: **2500000**

Update kernel:

- Shared memory atomics: **150000**
- Global atomics (with 256 threads/block): **29400**



Exercise 4: Memory Optimization

Question 1: The centroids are read by every thread. How could you optimize this access pattern?

Your answer: *centroids 会被所有线程反复读取，可以把 centroids 值存在 shared memory 或 constant memory 中，每个 block 先把 centroids_x 和 centroids_y 加载到 shared memory，然后 block 内所有线程复用，这样可以减少对 global memory 的重复读取。*

Question 2: If $k = 1000$, what would be the impact on cache performance?

Your answer: 如果 $k = 1000$ ，每个线程都要读取 1000 个 centroids 并计算 1000 次距离。centroid 数据量变大后，cache 命中率可能下降，global memory 访问压力增加，每个线程的计算量也显著增加，因此 assignment kernel 会变慢，cache performance 会变差。_____



Exercise 5: Convergence Analysis

Question 1: What is the convergence threshold in this implementation?

Your answer: 收敛阈值是 $1.0e-8$ 。

判断条件是: $dx dx + dy dy < 1.0e-8f$ _____

Question 2: Why is it important to check convergence rather than always running `max_iterations`?

Your answer: 检查 convergence 可以避免总是运行 `max_iterations`。如果 centroids 已经基本不再移动，继续迭代只会浪费计算时间。提前终止可以减少 kernel 调用次数，提高整体效率。_____

Question 3: How does the double-buffering (swapping `prev` and `new` centroids) help?

Your answer: double-buffering 使用 `prev` centroids 和 `new` centroids 两组数组交替保存结果。这样每一轮可以用 `prev` centroids 做分配，同时把重新计算的结果写入 `new` centroids，避免读写同一数组造成数据覆盖或不一致。每轮结束后交换两组指针即可。_____